

VAASENK

Role-Based Feature Flow Specification

Student Side | Teacher Side | Admin Side

Version 0.1 - Product Planning Draft

Product	Vaasenk - Classroom productivity and AI teaching assistant platform
Target market	Schools, colleges, coaching centers, and institutions in India
Primary roles	Admin, Teacher, Student
Core stack	Next.js frontend, Node.js backend, PostgreSQL database, React Native mobile app
Primary MVP promise	Teach more. Copy less. Reduce classroom note-copying time and teacher preparation burden.

1. Document Control

- This document expands the Vaasenk PRD into role-wise feature implementation specifications.
- Every feature uses the same structure: Feature, Purpose, User story, Acceptance criteria, UI behavior, Backend requirement, Edge cases, Error states, Empty states, Permissions, and Success criteria.
- The document is intended for frontend design, backend API planning, QA test case creation, and phased product execution.

Version	Date	Owner	Notes
0.1	May 2026	Vaasenk Product Team	Initial role-wise feature flow and acceptance specification.

2. Locked Product Decisions

- Vaasenk will serve schools, colleges, and coaching centers with structured classrooms, not open chat groups.
- Admin/institution setup owns syllabus upload, sample question paper upload, academic structure, and AI knowledge base configuration.
- Teachers receive ready classrooms and teacher-only AI tools instead of uploading syllabus repeatedly.
- Students get fast note access, bookmarks, downloads/offline access, doubts, notifications, and assignments, but no AI chatbot in MVP.
- Teacher AI chatbot is initially teacher-only and grounded in classroom-mapped syllabus and sample papers.
- AI question paper generation uses both syllabus and sample paper patterns to avoid missing important exam areas.
- One subscription covers the institution according to plan limits; pricing is not per classroom.

3. Scope Summary by Role

Role	Primary device	Main jobs	MVP focus
Student	Mobile first, web optional	Join classroom, view notes, bookmark, download, ask doubts, receive updates.	Fast, distraction-free learning material access.
Teacher	Mobile + web	Upload board notes, manage classroom content, generate question papers, use teacher-only AI assistant.	Reduce classroom note-copying and teacher preparation burden.
Admin	Web first	Institution setup, users, classrooms, syllabus, sample papers, AI readiness, billing, analytics.	Centralized control and scalable onboarding.

4. Architecture Assumptions for All Features

- Frontend web app: Next.js with TypeScript and Tailwind CSS / ShadCN UI.
- Mobile app: React Native or Expo with TypeScript.
- Backend: Node.js with Express.js or NestJS.
- Database: PostgreSQL, with pgvector recommended for RAG/vector search if the team wants AI knowledge storage inside Postgres.
- ORM: Prisma or Drizzle recommended for type-safe database access.
- Background jobs: Redis + BullMQ for OCR, AI indexing, embedding, export generation, and notifications.
- File storage: S3-compatible storage such as AWS S3, Cloudflare R2, or similar, accessed through signed URLs.
- AI workflow: Node.js backend should call Flowise, Langflow, or a custom RAG service internally. Frontend/mobile should never call Flowise/Langflow directly.

- Every table must include institution_id where multi-tenant isolation is required.

5. High-Level Role Flows

5.1 Student Primary Flow

1. Student logs in or receives account from institution.
2. Student joins classroom using code or admin assignment.
3. Student opens Home and selects a classroom.
4. Student views uploaded notes/materials.
5. Student bookmarks or downloads important notes.
6. Student asks doubt if clarification is needed.
7. Student receives notifications for new notes, replies, assignments, and announcements.

5.2 Teacher Primary Flow

8. Teacher logs in and sees assigned classrooms.
9. Teacher opens classroom and uploads board photo/PDF/text note.
10. Students receive note instantly and save time in class.
11. Teacher opens AI Question Paper Generator when exam preparation is needed.
12. System uses admin-uploaded syllabus and sample papers to generate a draft paper.
13. Teacher edits, exports, and saves the paper.
14. Teacher uses classroom AI chatbot for syllabus-based help and planning.

5.3 Admin Primary Flow

15. Admin sets up institution profile and subscription.
16. Admin creates academic structure: board, class, section, subject, academic year.
17. Admin imports/creates teachers and students.
18. Admin uploads syllabus PDFs and sample question papers once.
19. Admin creates classrooms, maps teachers, syllabus, and sample papers.
20. AI processing indexes documents and provisions teacher-only classroom AI assistants.
21. Admin monitors usage, billing, analytics, roles, and system health.

6. Feature Inventory

6.1 Student Feature Inventory

No.	Feature	Phase	Primary value
1	Student authentication and role-based entry	MVP	Allow students to securely log in and land only inside the student experience without seeing teacher or admin tools.
2	Join classroom using class code	MVP	Let students join the correct classroom using a secure code shared by teacher/admin.
3	Student Home dashboard	MVP	Give students a quick overview of recent notes, upcoming work, notifications, and joined classrooms.

4	My Classrooms and classroom detail	MVP	Organize all joined classrooms and provide classroom-specific notes, announcements, doubts, and assignments.
5	Notes feed	MVP	Show uploaded classroom notes in a clean, chronological, searchable feed.
6	Note detail viewer	MVP	Let students clearly view board photos, PDFs, text notes, and attached learning resources.
7	Search and filter notes	Phase 2, UI-ready in MVP	Help students find old notes quickly using title, tags, subject, date, and later OCR text extracted from images/PDFs.
8	Bookmarks	MVP	Allow students to save important notes for quick revision.
9	Downloads and offline access	Phase 2, UI-ready in MVP	Let students save notes for low-network situations common in Indian school/college environments.
10	Ask doubt on note	Phase 2	Let students ask contextual doubts attached to a specific note without disturbing the whole class.
11	Assignments and quizzes access	Phase 2	Allow students to see assigned work and complete simple quizzes/practice tasks linked to classroom content.
12	Student notifications	MVP	Notify students about new notes, replies, assignments, and important classroom updates.
13	Student profile and settings	MVP	Let students manage basic personal information, app preferences, security, and logout.
14	Student AI study assistant	Future phase, not MVP	Later provide students a safe AI assistant based on classroom syllabus and approved notes after teacher/admin controls are mature.

6.2 Teacher Feature Inventory

No.	Feature	Phase	Primary value
1	Teacher authentication and	MVP	Give teachers a secure

	role-based dashboard		login and a teacher-only dashboard with classroom, note, AI, and doubt tools.
2	Teacher classroom list and classroom detail	MVP	Let teachers manage daily classroom activity from a structured subject-wise space.
3	Quick board photo upload	MVP	Enable the core VaasenK value: teacher captures board notes and publishes them instantly to students.
4	Upload PDFs, text notes, and resources	MVP	Allow teachers to share more than board images: PDFs, worksheets, typed notes, links, and reference materials.
5	Note preview, crop, rotate, and quality check	Phase 2, UI-ready in MVP	Improve readability of board photos before they reach students.
6	Tag and organize notes	MVP	Make notes easy to find by type, importance, chapter, and exam relevance.
7	Basic annotation on uploaded notes	Phase 2	Let teachers highlight, mark, or add clarifications on uploaded images/PDF pages.
8	AI Question Paper Generator	MVP strong differentiator	Reduce teacher workload by generating question papers from admin-uploaded syllabus and sample paper patterns.
9	Generated paper preview, edit, download, and share	MVP	Allow teachers to verify and customize AI-generated papers before using them officially.
10	Teacher-only classroom AI chatbot	MVP / Phase 1.5	Give each classroom its own AI assistant grounded in the admin-uploaded syllabus and sample papers, initially only for teachers.
11	Manage student doubts	Phase 2	Help teachers respond to student questions in an organized queue instead of scattered chats.
12	Create assignments and quizzes	Phase 2	Let teachers create practice tasks and quick checks connected to classroom lessons.
13	Teacher engagement analytics	Phase 2	Show teachers whether students are accessing notes, downloading materials, and responding to assignments.
14	Teacher notifications and	MVP	Keep teachers aware of

	reminders		student joins, doubts, AI job completion, upload success, and pending classroom activity.
15	Teacher profile and settings	MVP	Allow teachers to manage account details and teaching preferences.
16	Teacher collaboration and reusable paper library	Phase 3	Allow teachers in the same institution to reuse approved generated papers, templates, and teaching resources.

6.3 Admin Feature Inventory

No.	Feature	Phase	Primary value
1	Admin authentication and institution dashboard	MVP	Give school/college admins a secure control center for users, classrooms, syllabus, billing, AI setup, and analytics.
2	Institution onboarding and profile setup	MVP	Create the institution workspace with academic metadata, contact details, branding, and subscription mapping.
3	Subscription and billing management	MVP	Enable recurring monetization at institution level with one payment covering unlimited classrooms under plan limits.
4	Teacher and student user management	MVP	Allow admins to add, invite, edit, deactivate, and manage teachers/students at institution level.
5	Bulk import users	Phase 2, optional MVP for schools	Reduce onboarding effort by importing many teachers/students using CSV/Excel templates.
6	Academic structure setup	MVP	Model school/college structure such as board, class, section, subject, department, academic year, and term.
7	Classroom creation, assignment, and syllabus mapping	MVP	Create classroom rooms with assigned teachers/students and mapped syllabus so teachers do not handle syllabus upload repeatedly.
8	Syllabus repository upload and management	MVP	Centralize syllabus PDFs at admin level for Samacheer Kalvi, CBSE, custom school syllabus,

			college subjects, and coaching courses.
9	Sample question paper and exam pattern library	MVP strong differentiator	Allow admins to upload sample papers so AI generation follows real exam style and important question patterns.
10	AI document processing and indexing monitor	MVP / Phase 1.5	Let admins track whether uploaded syllabi and sample papers are ready for AI chatbot and question generation.
11	Classroom AI bot provisioning and configuration	MVP / Phase 1.5	Automatically create or enable teacher-only AI assistants for classrooms with ready syllabus/sample-paper knowledge bases.
12	AI usage controls and quota management	MVP / Phase 1.5	Protect costs and ensure fair usage for question generation, chatbot, OCR, and embeddings.
13	Content storage and moderation management	Phase 2	Help admins monitor storage, remove inappropriate/old content, and manage institution file usage.
14	School-wide announcements	Phase 2	Allow admins to send official institution-level announcements to teachers, students, or selected classes.
15	Institution analytics and reports	Phase 2	Give admins measurable proof of Vaasenk usage, adoption, and learning workflow impact.
16	Roles and permissions management	MVP	Control what admins, teachers, students, and future roles can see and do.
17	Audit logs	Phase 2	Maintain accountability for sensitive actions such as uploads, deletes, AI generation, permission changes, and billing updates.
18	Branding and institution settings	Phase 2	Let institutions configure basic branding, academic preferences, notification rules, and app policies.
19	Data export and backup	Phase 3	Allow institutions to export important data for compliance, migration, backup, or academic records.
20	Support and onboarding	MVP	Guide admins through

7. Student Side - Detailed Feature Specifications

7.1. Student authentication and role-based entry

Feature	Student authentication and role-based entry
Release phase	MVP

Purpose:

Allow students to securely log in and land only inside the student experience without seeing teacher or admin tools.

Detailed flow:

- Student opens app or web portal.
- Student enters phone/email and password/OTP depending on final auth method.
- System validates credentials and role.
- If role is student, redirect to Student Home.
- If account is inactive or institution subscription is expired, show a blocked access message.

User story:

As a student, I want to log in and directly see my classrooms so I can access notes without confusion.

Acceptance criteria:

- Student can log in using valid credentials.
- Student cannot access teacher/admin URLs or screens.
- Session persists until logout/token expiry.
- Forgot password flow is visible.
- Invalid login does not reveal whether email/phone exists.

UI behavior:

- Mobile: simple login form with institution/app branding, large CTA, password visibility toggle.
- Web: centered login card with role-safe routing after login.
- Show loading state after submit and disable duplicate taps.

Backend requirement:

- Auth service with JWT/session tokens.
- Users table with role=student, institution_id, status.
- Password hashing or OTP provider integration.
- Middleware to enforce student routes.
- Refresh token/session expiry handling.

Edge cases:

- Student belongs to multiple institutions.
- Student account created but not assigned to any classroom.
- Institution subscription expired.
- Student changes device.
- Student uses teacher invite link accidentally.

Error states:

- Invalid credentials.
- Account inactive.
- Network failure.
- Session expired.
- Institution access suspended.

Empty states:

- No classrooms yet: show Join Classroom CTA.
- No internet: show retry and cached login state if available.

Permissions:

- Student can view only own profile, joined classrooms, allowed notes, bookmarks, downloads, doubts, and assignments.
- Student cannot create classrooms, upload official notes, generate question papers, upload syllabus, or access teacher chatbot in MVP.

Success criteria:

- Student reaches home screen in under 3 seconds on stable connection.
- Zero teacher/admin controls visible to student.
- Login failure messages are clear and safe.

7.2. Join classroom using class code

Feature	Join classroom using class code
Release phase	MVP

Purpose:

Let students join the correct classroom using a secure code shared by teacher/admin.

Detailed flow:

- Student taps Join Classroom.
- Student enters classroom code.
- System validates code, institution, classroom status, and student eligibility.
- Student confirms classroom details.
- System adds student to classroom.
- Student sees the classroom in My Classrooms.

User story:

As a student, I want to join a classroom using a code so I can access the correct notes and announcements.

Acceptance criteria:

- Valid active class code joins the student.
- Already joined classroom shows an informative message.
- Expired/invalid code is rejected.
- Student can see classroom name, subject, teacher before confirming.
- Joining triggers optional notification to teacher/admin.

UI behavior:

- Single input field for code with auto-uppercase.
- Confirmation bottom sheet/modal before joining.
- Success animation and View Classroom CTA.
- Display helpful text: Ask your teacher for the class code.

Backend requirement:

- Classrooms table with join_code, code_expiry, status.
- Classroom_members table with student_id, classroom_id, role=student, status.
- Endpoint: POST /classrooms/join.
- Uniqueness constraint to prevent duplicate membership.

Edge cases:

- Code typed with spaces/lowercase.
- Code belongs to another institution.
- Student removed earlier and tries to rejoin.
- Classroom archived.
- Maximum student limit reached, if plan enforces limits.

Error states:

- Invalid code.
- Expired code.
- Classroom not found.
- Already joined.
- Server timeout.

Empty states:

- Before joining: show illustration and explanation.
- No available classroom: show contact teacher/admin message.

Permissions:

- Student can join only with valid code.
- Admin/teacher can disable or rotate codes.
- Student cannot view classroom before membership is approved/created.

Success criteria:

- Join flow completed in less than 30 seconds.
- Low support requests caused by wrong classroom joins.
- Accidental duplicate joins prevented.

7.3. Student Home dashboard

Feature	Student Home dashboard
Release phase	MVP

Purpose:

Give students a quick overview of recent notes, upcoming work, notifications, and joined classrooms.

Detailed flow:

- Student logs in.
- Home loads recent notes from joined classrooms.
- Student sees quick cards: My Classrooms, Bookmarks, Downloads, Notifications.
- Student taps a recent item or classroom.

User story:

As a student, I want one simple home screen so I can quickly continue studying from the latest materials.

Acceptance criteria:

- Shows recently uploaded notes sorted by date.
- Shows joined classrooms count.
- Shows unread notification count.
- Provides Join Classroom action.
- Works even if the student has zero classrooms.

UI behavior:

- Mobile: vertical feed with top greeting, search bar, recent cards, bottom navigation.
- Web: dashboard cards plus recent notes list.
- Use clear labels and avoid teacher-only actions.

Backend requirement:

- Endpoint: GET /students/home.
- Aggregates classroom memberships, notes, notifications, bookmarks.
- Pagination for recent items.
- Cacheable response for mobile performance.

Edge cases:

- Student has too many classrooms.
- Multiple notes uploaded at same time.
- Deleted classroom notes should disappear.

- Student has blocked/removed membership.

Error states:

- Unable to load dashboard.
- Partial data failure for notifications or notes.
- Unauthorized token.

Empty states:

- No classrooms: show Join Classroom CTA.
- No notes: show Waiting for your teacher to upload notes.
- No notifications: show You are all caught up.

Permissions:

- Student can see only data from joined active classrooms.
- No access to draft/unpublished notes.

Success criteria:

- Students can find latest material without asking teacher.
- Home screen is understandable within first use.
- Dashboard API remains fast under normal class size.

7.4. My Classrooms and classroom detail

Feature	My Classrooms and classroom detail
Release phase	MVP

Purpose:

Organize all joined classrooms and provide classroom-specific notes, announcements, doubts, and assignments.

Detailed flow:

- Student taps My Classrooms.
- System lists all joined classrooms.
- Student selects a classroom.
- Classroom Detail opens with tabs/sections: Notes, Announcements, Doubts, Assignments.
- Student opens a note or starts a doubt.

User story:

As a student, I want all my subjects organized separately so I do not mix notes from different classes.

Acceptance criteria:

- Classroom list shows name, subject, teacher, latest activity.
- Classroom detail shows only content belonging to that classroom.
- Archived classrooms are visually marked or hidden based on policy.
- Student can search/filter within a classroom.

UI behavior:

- Classroom cards with subject color/icon.
- Tabs on detail screen.
- Sticky classroom header with teacher name and subject.
- Use date grouping for notes.

Backend requirement:

- Classrooms, classroom_members, notes, announcements, assignments tables.
- Endpoint: GET /students/classrooms and GET /classrooms/:id/student-view.
- Access control based on membership.

Edge cases:

- Teacher changes classroom name.
- Student removed from classroom while viewing.
- Classroom has no teacher assigned.

- Classroom archived after exam.

Error states:

- Classroom not found.
- Access denied.
- Content failed to load.
- Network retry needed.

Empty states:

- No classrooms joined.
- No notes in selected classroom.
- No assignments/doubts yet.

Permissions:

- Student can view only joined classrooms.
- Student cannot edit classroom metadata.
- Student cannot see teacher-only analytics or AI chatbot in MVP.

Success criteria:

- Students can reach any note in two to three taps.
- Subject separation reduces confusion.
- Classroom detail remains usable with hundreds of notes.

7.5. Notes feed

Feature	Notes feed
Release phase	MVP

Purpose:

Show uploaded classroom notes in a clean, chronological, searchable feed.

Detailed flow:

- Student opens Notes tab or global Notes feed.
- System loads notes from joined classrooms.
- Student filters by subject, tag, date, or teacher.
- Student taps a note to view details.

User story:

As a student, I want to see notes neatly arranged so I can revise without scrolling through chat messages.

Acceptance criteria:

- Notes show title, subject, teacher, upload date, file type, tags.
- Feed supports pagination/infinite scroll.
- Notes can be filtered by classroom and tag.
- Deleted/unpublished notes do not appear.

UI behavior:

- Card/list layout with thumbnail for image/PDF.
- Important tag highlighted.
- Bookmark and download quick actions.
- Skeleton loading while fetching.

Backend requirement:

- Notes table with classroom_id, teacher_id, title, description, file_url, file_type, tags, status.
- Endpoint: GET /students/notes.
- Search/filter query parameters.
- Signed URLs for file access.

Edge cases:

- Huge number of notes.

- Teacher uploads duplicate title.
- Note has multiple attachments.
- Thumbnail generation pending.
- PDF too large for preview.

Error states:

- Notes failed to load.
- File unavailable.
- Unsupported preview format.
- Permission revoked.

Empty states:

- No notes: Waiting for teacher upload.
- No filtered results: Clear filters CTA.

Permissions:

- Student can view only published notes in active joined classrooms.
- Download permission depends on note settings.

Success criteria:

- Students stop asking for board photos in external apps.
- Average note discovery time is reduced.
- Feed is readable and clutter-free.

7.6. Note detail viewer

Feature	Note detail viewer
Release phase	MVP

Purpose:

Let students clearly view board photos, PDFs, text notes, and attached learning resources.

Detailed flow:

- Student taps a note card.
- Note detail opens.
- System loads title, description, attachment preview, teacher, date, tags.
- Student can zoom image/PDF, download, bookmark, or ask doubt.

User story:

As a student, I want to open a note and read it clearly so I do not need to copy board content manually.

Acceptance criteria:

- Images support pinch zoom on mobile.
- PDFs are scrollable.
- Text notes are readable with good line spacing.
- Download/bookmark actions work.
- File preview gracefully handles unsupported formats.

UI behavior:

- Mobile: full-screen viewer with sticky top bar and bottom actions.
- Web: split layout if needed: preview left, metadata/actions right.
- Dark overlay for image viewing.

Backend requirement:

- Endpoint: GET /notes/:id.
- Signed file delivery via object storage.
- Download tracking endpoint.
- Preview/thumbnail metadata.
- Optional CDN integration.

Edge cases:

- Very large image.
- Low-quality board photo.
- File deleted from storage but DB row exists.
- Student opens note while offline.
- Teacher updates note metadata.

Error states:

- Preview failed.
- Download failed.
- File expired.
- Access denied.
- Storage service unavailable.

Empty states:

- Attachment unavailable: show fallback with message.
- No description: hide description area.

Permissions:

- Student must be classroom member.
- Download allowed only when teacher/admin settings permit.
- No edit/delete controls for student.

Success criteria:

- Notes are readable on small screens.
- Students can save or access note without external messaging apps.
- File view errors are recoverable.

7.7. Search and filter notes

Feature	Search and filter notes
Release phase	Phase 2, UI-ready in MVP

Purpose:

Help students find old notes quickly using title, tags, subject, date, and later OCR text extracted from images/PDFs.

Detailed flow:

- Student taps search.
- Student types keyword or selects filters.
- System searches accessible notes metadata and OCR text when available.
- Results appear grouped by classroom/date.
- Student opens matching note.

User story:

As a student, I want to search for a topic like Photosynthesis so I can quickly find the correct note before exams.

Acceptance criteria:

- Search works by title, description, tag, classroom, teacher.
- OCR text is included once processing is available.
- Filters can be combined.
- No results state is clear.
- Search is access-controlled.

UI behavior:

- Prominent search bar on Home/Notes.
- Filter chips for subject/date/tag/file type.
- Highlight matching text in results when possible.
- Recent searches optional.

Backend requirement:

- Postgres full-text search and/or vector/OCR index.
- OCR processing jobs for images/PDFs.
- Endpoint: GET /notes/search?q=&filters.
- Store extracted_text and processing_status.

Edge cases:

- Misspelled keyword.
- OCR confidence low.
- Multiple languages/Tamil-English mix.
- Old images without OCR yet.
- Search across many classrooms.

Error states:

- Search service timeout.
- OCR index unavailable.
- Invalid filter values.

Empty states:

- No results: suggest checking spelling or filters.
- OCR processing pending: show Search will improve after processing.

Permissions:

- Search results limited to student membership.
- No access to hidden/draft files.

Success criteria:

- Students can find notes without manual scrolling.
- Search returns relevant results quickly.
- OCR-based recall improves revision value.

7.8. Bookmarks

Feature	Bookmarks
Release phase	MVP

Purpose:

Allow students to save important notes for quick revision.

Detailed flow:

- Student taps bookmark on note card or detail.
- System saves bookmark.
- Student opens Bookmarks screen.
- Bookmarked notes are listed by classroom/date.
- Student can remove bookmark.

User story:

As a student, I want to bookmark important notes so I can revise them quickly before exams.

Acceptance criteria:

- Bookmark toggle works instantly with optimistic UI.
- Bookmarked notes appear in Bookmarks list.
- Removing bookmark updates all places.
- Bookmark is unique per student/note.

UI behavior:

- Outline bookmark icon becomes filled.
- Small toast: Saved to bookmarks.
- Bookmarks screen available from bottom nav/profile.

Backend requirement:

- Bookmarks table with user_id, note_id, created_at.
- Endpoints: POST/DELETE /notes/:id/bookmark and GET /students/bookmarks.
- Unique constraint user_id + note_id.

Edge cases:

- Note deleted after bookmarked.
- Student removed from classroom.
- Bookmark tapped repeatedly.
- Offline bookmark queued for sync.

Error states:

- Bookmark save failed.
- Bookmark load failed.
- Access revoked.

Empty states:

- No bookmarks: show Save important notes here.

Permissions:

- Student can bookmark only accessible notes.
- Bookmark private to student unless analytics later tracks aggregate counts.

Success criteria:

- Higher repeat access to important notes.
- Students can build personal revision list.

7.9. Downloads and offline access

Feature	Downloads and offline access
Release phase	Phase 2, UI-ready in MVP

Purpose:

Let students save notes for low-network situations common in Indian school/college environments.

Detailed flow:

- Student taps download/offline toggle.
- App downloads file securely.
- Downloaded note appears in Downloads.
- Student opens downloaded note without internet.
- When online, app validates access and syncs metadata.

User story:

As a student, I want to access notes offline so I can study even when internet is weak.

Acceptance criteria:

- Download progress is visible.
- Downloaded files open offline.
- Student can remove downloads to save space.
- App respects teacher/admin download restrictions.
- Stale/revoked notes are handled on next sync.

UI behavior:

- Download icon with progress ring.
- Offline badge on saved notes.
- Downloads screen with storage used.
- Warning for large files.

Backend requirement:

- Mobile local encrypted storage/cache.

- Download permission flag in note settings.
- Endpoint for signed URLs.
- Sync job to validate membership and file version.
- Web can offer normal download without offline cache.

Edge cases:

- Insufficient device storage.
- Large PDF on low network.
- Student loses membership after download.
- File version updated by teacher.
- Device clock mismatch.

Error states:

- Download failed.
- Storage full.
- Offline file corrupted.
- Access revoked.

Empty states:

- No downloads: show Download notes to study offline.

Permissions:

- Student can download only permitted published notes.
- Offline cached content should be protected from casual unauthorized access where feasible.

Success criteria:

- Offline opening success rate high.
- Reduced dependency on WhatsApp forwarding.
- Students trust Vaasenk during poor connectivity.

7.10. Ask doubt on note

Feature	Ask doubt on note
Release phase	Phase 2

Purpose:

Let students ask contextual doubts attached to a specific note without disturbing the whole class.

Detailed flow:

- Student opens a note.
- Student taps Ask Doubt.
- Student types question and optionally attaches image.
- Teacher receives doubt in classroom doubt queue.
- Teacher replies.
- Student receives notification and views reply thread.

User story:

As a student, I want to ask a doubt on a specific note so the teacher understands the exact context.

Acceptance criteria:

- Student can create doubt under note/classroom.
- Teacher can reply.
- Thread remains attached to note.
- Student can mark resolved.
- Spam/abuse controls exist.

UI behavior:

- Ask Doubt CTA on note detail.
- Thread UI similar to clean comments, not chaotic chat.
- Status labels: Open, Answered, Resolved.

- Teacher replies visually distinct.

Backend requirement:

- Doubts table with student_id, teacher_id/classroom_id, note_id, message, status.
- Doubt_replies table.
- Notification trigger on new doubt/reply.
- Moderation/report option later.

Edge cases:

- Multiple students ask same doubt.
- Teacher unavailable.
- Student edits/deletes doubt.
- Doubt on deleted note.
- Abusive content.

Error states:

- Message failed to send.
- Attachment upload failed.
- Thread access denied.
- Teacher reply failed.

Empty states:

- No doubts yet: Ask your first doubt.
- Teacher view: No pending doubts.

Permissions:

- Student can create/view own doubts and public class doubts only if configured.
- Teacher sees doubts for their classrooms.
- Admin can audit if needed.

Success criteria:

- Doubts move out of WhatsApp.
- Teachers get better context.
- Students receive answers without interrupting class flow.

7.11. Assignments and quizzes access

Feature	Assignments and quizzes access
Release phase	Phase 2

Purpose:

Allow students to see assigned work and complete simple quizzes/practice tasks linked to classroom content.

Detailed flow:

- Teacher publishes assignment/quiz.
- Student receives notification.
- Student opens Assignments tab.
- Student views instructions/due date.
- Student submits answer/file or completes quiz.
- System records submission.

User story:

As a student, I want assignments and quizzes in the same classroom app so I do not miss work.

Acceptance criteria:

- Student can see pending/completed/overdue assignments.
- Due dates are visible.
- Submission status is clear.
- Quiz result shown based on teacher settings.
- Late submission policy visible.

UI behavior:

- Assignment cards with due date and status badge.
- Submission screen with upload/text answer.
- Quiz UI with one question per screen or list mode.
- Clear Submit confirmation.

Backend requirement:

- Assignments, assignment_submissions, quizzes, quiz_questions, quiz_attempts tables.
- File upload storage.
- Deadline validation.
- Notification scheduler.

Edge cases:

- Late submissions.
- Reattempts allowed/not allowed.
- Network loss during quiz.
- Teacher updates assignment after publish.
- Student submits wrong file.

Error states:

- Submission failed.
- Deadline passed.
- File too large.
- Quiz attempt expired.

Empty states:

- No assignments: No work assigned yet.
- No completed items: Completed assignments appear here.

Permissions:

- Student can view/submit only assigned classroom work.
- Student cannot view other students submissions.
- Teacher/admin can view submissions.

Success criteria:

- Reduced missed assignments.
- Clear status tracking.
- Teachers can monitor completion.

7.12. Student notifications

Feature	Student notifications
Release phase	MVP

Purpose:

Notify students about new notes, replies, assignments, and important classroom updates.

Detailed flow:

- Event occurs: new note, announcement, doubt reply, assignment.
- Backend creates notification.
- Mobile push/in-app notification sent where enabled.
- Student opens notification center.
- Student taps item and is deep-linked to the relevant screen.

User story:

As a student, I want timely alerts so I do not miss important notes or teacher replies.

Acceptance criteria:

- Notifications are grouped by classroom/event.
- Unread/read states exist.

- Tapping notification opens correct content.
- Student can manage notification preferences.
- No duplicate spam for same event.

UI behavior:

- Bell icon with unread count.
- Notification list grouped by Today/Older.
- Deep-link support.
- Preference toggles in settings.

Backend requirement:

- Notifications table with recipient_id, type, payload, read_at.
- Push provider integration for mobile.
- Event-based notification service.
- Rate limiting and batching logic.

Edge cases:

- Teacher uploads many notes quickly.
- Student disabled push permission.
- Content deleted before notification opened.
- Duplicate event retries.

Error states:

- Push delivery failed.
- Notification content unavailable.
- Permission denied for push.

Empty states:

- No notifications: You are all caught up.

Permissions:

- Student receives only own classroom notifications.
- Sensitive content should not be exposed fully in push preview if configured.

Success criteria:

- Students respond faster to new notes and replies.
- Low duplicate notification complaints.

7.13. Student profile and settings

Feature	Student profile and settings
Release phase	MVP

Purpose:

Let students manage basic personal information, app preferences, security, and logout.

Detailed flow:

- Student opens Profile.
- Student views name, email/phone, institution, joined classrooms.
- Student edits allowed fields.
- Student manages notifications/theme/download settings.
- Student can logout.

User story:

As a student, I want to manage my account and preferences so the app feels personal and controlled.

Acceptance criteria:

- Student can view profile.
- Editable fields are controlled by institution policy.
- Password change/logout works.

- Notification preferences save.
- Theme setting applies consistently.

UI behavior:

- Simple profile header.
- Settings list with grouped sections.
- Clear logout button.
- Danger actions separated.

Backend requirement:

- Users table update endpoint.
- Preferences table/json column.
- Auth password update endpoint.
- Device tokens management.

Edge cases:

- Admin-locked profile fields.
- Student changes phone/email requiring verification.
- Multiple devices.
- Logout while offline.

Error states:

- Profile update failed.
- Verification failed.
- Password mismatch.
- Settings save failed.

Empty states:

- Missing profile photo: initials/avatar placeholder.

Permissions:

- Student can edit only allowed own profile fields.
- Cannot change role/institution from profile.

Success criteria:

- Settings reduce support tickets.
- Profile stays accurate.
- Logout/security flows are reliable.

7.14. Student AI study assistant

Feature	Student AI study assistant
Release phase	Future phase, not MVP

Purpose:

Later provide students a safe AI assistant based on classroom syllabus and approved notes after teacher/admin controls are mature.

Detailed flow:

- Admin enables student AI for institution/classroom.
- Teacher approves sources and mode.
- Student opens AI Study Assistant.
- Student asks question.
- AI answers only from approved classroom material with citations/links to notes.
- Unsafe/off-topic prompts are blocked.

User story:

As a student, I want to ask study questions from my syllabus so I can revise independently.

Acceptance criteria:

- Feature disabled by default in MVP.
- Answers must be grounded in approved material.
- AI should say when answer is not available.
- Usage limits are enforced.
- Teacher/admin can review usage if policy allows.

UI behavior:

- Separate AI tab with clear educational disclaimer.
- Source chips linked to notes/syllabus.
- Feedback buttons.
- Daily usage indicator.

Backend requirement:

- RAG pipeline using classroom knowledge base.
- Prompt guardrails.
- Usage tracking.
- Student-safe retrieval scope.
- Moderation and logging policy.

Edge cases:

- Student asks exam answer directly.
- Student asks unrelated questions.
- Hallucination risk.
- Poor OCR/source quality.
- High usage cost.

Error states:

- AI unavailable.
- No sources indexed.
- Usage limit reached.
- Unsafe request blocked.

Empty states:

- AI not enabled: Coming later / Ask your teacher.
- No indexed content: Study assistant not ready.

Permissions:

- Student AI only when enabled by admin/teacher.
- No private teacher-only sample paper answer keys exposed unless approved.

Success criteria:

- Students receive useful grounded help.
- Teachers trust AI safety.
- No leakage of confidential teacher materials.

8. Teacher Side - Detailed Feature Specifications

8.1. Teacher authentication and role-based dashboard

Feature	Teacher authentication and role-based dashboard
Release phase	MVP

Purpose:

Give teachers a secure login and a teacher-only dashboard with classroom, note, AI, and doubt tools.

Detailed flow:

- Teacher logs in.
- System validates teacher role and institution status.
- Teacher lands on Teacher Home.
- Dashboard shows classrooms, recent uploads, pending doubts, AI tools, and quick upload actions.

User story:

As a teacher, I want to log in and immediately access my classes and teaching tools.

Acceptance criteria:

- Teacher sees teacher dashboard only.
- Student/admin-only screens are hidden.
- Teacher can access assigned classrooms.
- Teacher-only AI tools are visible.
- Inactive teachers are blocked.

UI behavior:

- Dashboard with cards: My Classrooms, Upload Note, Generate Question Paper, AI Chatbot, Pending Doubts.
- Mobile has prominent quick camera upload.
- Web has dashboard layout with side navigation.

Backend requirement:

- Users table role=teacher.
- Teacher_classroom_assignments table.
- Role middleware.
- Institution subscription checks.
- Dashboard aggregation endpoint.

Edge cases:

- Teacher teaches multiple institutions.
- Teacher not assigned to classroom yet.
- Subscription expired.
- Teacher role changed by admin while logged in.

Error states:

- Access denied.
- No assigned classrooms.
- Session expired.
- Dashboard load failed.

Empty states:

- No classrooms: Contact admin or create/request classroom depending policy.
- No pending doubts: show clear state.

Permissions:

- Teacher can manage assigned classrooms and own content.
- Teacher cannot upload syllabus/sample papers unless admin grants permission.
- Teacher cannot access billing/admin settings.

Success criteria:

- Teachers reach upload/generate flows quickly.

- No student/admin controls leak into teacher UI.
- Teacher onboarding feels simple.

8.2. Teacher classroom list and classroom detail

Feature	Teacher classroom list and classroom detail
Release phase	MVP

Purpose:

Let teachers manage daily classroom activity from a structured subject-wise space.

Detailed flow:

- Teacher opens My Classrooms.
- System lists assigned/created classrooms.
- Teacher selects classroom.
- Detail page shows Notes, Students, Doubts, Assignments, AI Assistant, Generated Papers.
- Teacher performs actions for that classroom.

User story:

As a teacher, I want each subject/class separated so I can upload and manage materials properly.

Acceptance criteria:

- Teacher sees only assigned classrooms.
- Classroom detail has clear actions.
- Student count and latest activity visible.
- Archived classrooms are separated.
- Teacher can copy/share join code if allowed.

UI behavior:

- Classroom cards with subject, section, students, latest upload.
- Tabs or segmented controls.
- Primary action: Upload Note.
- AI Assistant visible only if syllabus is indexed.

Backend requirement:

- Classrooms and assignments tables.
- Endpoint: GET /teachers/classrooms and GET /teachers/classrooms/:id.
- Join code read endpoint.
- Classroom membership counts.

Edge cases:

- Teacher assigned after classroom created.
- Classroom has no syllabus mapped.
- Students not joined yet.
- Classroom archived mid-session.

Error states:

- Classroom access denied.
- Classroom not found.
- Data load failure.

Empty states:

- No classrooms assigned.
- No students joined.
- No notes uploaded.

Permissions:

- Teacher can view/manage assigned classrooms.
- Admin can override.
- Students cannot access this management view.

Success criteria:

- Teachers understand class status at a glance.
- Daily classroom management requires fewer taps/clicks.

8.3. Quick board photo upload

Feature	Quick board photo upload
Release phase	MVP

Purpose:

Enable the core Vaasenk value: teacher captures board notes and publishes them instantly to students.

Detailed flow:

- Teacher opens classroom.
- Teacher taps Upload Note or camera shortcut.
- Teacher selects/takes photo.
- Teacher previews/crops/rotates if needed.
- Teacher adds title/description/tags.
- Teacher publishes.
- Students receive notification.

User story:

As a teacher, I want to quickly upload board photos so students can focus on listening instead of copying.

Acceptance criteria:

- Teacher can upload image from camera/gallery.
- Teacher must select classroom.
- Preview shown before publish.
- Title can be auto-filled but editable.
- Successful upload appears in classroom feed.

UI behavior:

- Mobile: camera-first flow with crop/retake options.
- Web: drag-and-drop upload plus file picker.
- Show upload progress and success toast.
- After upload: View Note / Upload Another.

Backend requirement:

- File upload endpoint with object storage.
- Notes table record.
- Thumbnail generation job.
- Notification event.
- OCR extraction job queued for Phase 2 search.

Edge cases:

- Poor internet during upload.
- Huge image file.
- Blurry board photo.
- Teacher selects wrong classroom.
- Duplicate upload.

Error states:

- Upload failed.
- File too large.
- Unsupported file type.
- Storage unavailable.
- Notification failed but upload succeeded.

Empty states:

- No classroom selected: prompt selection.

- No students joined: allow upload but show info.

Permissions:

- Only assigned teacher/admin can upload to classroom.
- Students cannot upload official notes unless explicitly allowed as contributor.

Success criteria:

- Teacher can publish note in under one minute.
- Students can view uploaded note immediately.
- Core classroom time-saving promise is fulfilled.

8.4. Upload PDFs, text notes, and resources

Feature	Upload PDFs, text notes, and resources
Release phase	MVP

Purpose:

Allow teachers to share more than board images: PDFs, worksheets, typed notes, links, and reference materials.

Detailed flow:

- Teacher taps Upload Material.
- Teacher chooses file/text/link type.
- Teacher adds title, description, tags.
- Teacher selects classroom.
- Teacher publishes.
- Material appears in feed.

User story:

As a teacher, I want to upload PDFs and typed notes so all classroom resources stay in one place.

Acceptance criteria:

- Supports PDF, image, text note, and optional link.
- File size/type limits enforced.
- Preview metadata shown before publish.
- Material can be tagged.
- Published material visible to students.

UI behavior:

- Upload type selector.
- Drag-and-drop on web.
- File preview card.
- Tag chips.
- Publish button disabled until required fields valid.

Backend requirement:

- Notes/materials table with file_type.
- Object storage and signed URLs.
- File validation service.
- Text notes stored as content body.
- Optional link validation.

Edge cases:

- PDF with many pages.
- Password-protected PDF.
- Unsupported file extension.
- External link unsafe.
- Teacher uploads wrong version.

Error states:

- Validation failed.

- PDF preview failed.
- Upload timeout.
- Unsafe link blocked.

Empty states:

- No file selected.
- No description: optional and hidden.

Permissions:

- Teacher can upload to assigned classrooms.
- Admin can remove/moderate content.
- Students can view/download based on settings.

Success criteria:

- All teaching material becomes centralized.
- Teachers do not rely on WhatsApp file forwarding.

8.5. Note preview, crop, rotate, and quality check

Feature	Note preview, crop, rotate, and quality check
Release phase	Phase 2, UI-ready in MVP

Purpose:

Improve readability of board photos before they reach students.

Detailed flow:

- Teacher captures/selects image.
- Preview opens.
- Teacher crops unwanted background, rotates, or retakes.
- System may warn if image is blurry/dark.
- Teacher confirms and publishes.

User story:

As a teacher, I want to clean up a board photo before posting so students can read it clearly.

Acceptance criteria:

- Crop/rotate available for images.
- Retake option available on mobile.
- Quality warning shown but does not always block.
- Original/compressed version stored based on policy.

UI behavior:

- Image editor with crop handles and rotate button.
- Quality alert: This photo may be hard to read.
- One-tap Continue Anyway.

Backend requirement:

- Client-side image processing where possible.
- Server-side compression and thumbnail job.
- Optional quality-check AI/computer vision later.
- Store processed file metadata.

Edge cases:

- Landscape/portrait rotation issue.
- Very dark board.
- Multiple board photos needed.
- Teacher accidentally crops content.

Error states:

- Processing failed.

- Image editor crash/retry.
- Upload after edit failed.

Empty states:

- No image selected.
- Image preview unavailable.

Permissions:

- Teacher controls edits before publish.
- Admin can set max file sizes.
- Students only see final published version.

Success criteria:

- Improved note readability.
- Lower student complaints about unclear notes.

8.6. Tag and organize notes

Feature	Tag and organize notes
Release phase	MVP

Purpose:

Make notes easy to find by type, importance, chapter, and exam relevance.

Detailed flow:

- Teacher uploads or edits a note.
- Teacher selects tags like Important, Homework, Exam, Revision.
- Teacher optionally maps note to chapter/topic.
- Students see tags in feed.
- Search/filter uses tags.

User story:

As a teacher, I want to tag notes so students can easily identify important materials.

Acceptance criteria:

- Predefined tags available.
- Custom tags optional by institution settings.
- Tags display on note cards.
- Filtering by tags works.
- Chapter/topic mapping possible when syllabus exists.

UI behavior:

- Tag chips selectable during upload.
- Important tag visually highlighted.
- Chapter dropdown appears if syllabus mapped.

Backend requirement:

- Tags column/join table.
- Classroom_topics derived from syllabus repository.
- Note_topic mapping.
- Filter endpoints.

Edge cases:

- Too many tags.
- Teacher misspells custom tag.
- Syllabus topics not indexed.
- Tag removed later.

Error states:

- Tag save failed.

- Topic mapping unavailable.
- Invalid custom tag.

Empty states:

- No tags: note still publishable.
- No syllabus topics: hide chapter mapping.

Permissions:

- Teacher/admin can define tags.
- Students can view/filter tags but not edit official tags.

Success criteria:

- Notes remain organized over a full academic year.
- Students can filter revision/exam notes quickly.

8.7. Basic annotation on uploaded notes

Feature	Basic annotation on uploaded notes
Release phase	Phase 2

Purpose:

Let teachers highlight, mark, or add clarifications on uploaded images/PDF pages.

Detailed flow:

- Teacher opens uploaded note.
- Teacher taps Annotate.
- Teacher adds highlight, arrow, text, or drawing.
- Teacher saves annotated version.
- Students see updated annotated note or version.

User story:

As a teacher, I want to mark important parts of a note so students know what to focus on.

Acceptance criteria:

- Teacher can draw/highlight on image.
- Annotated version is saved.
- Students see latest version.
- Version history optional.
- Annotation does not destroy original file without backup.

UI behavior:

- Simple toolbar: pen, highlighter, text, undo, save.
- Confirmation before replacing published note.
- Version label if edited.

Backend requirement:

- Image/PDF annotation storage strategy.
- Version table for notes.
- Object storage for annotated file.
- Notification optional on update.

Edge cases:

- Large PDF annotation.
- Teacher edits while students viewing.
- Accidental annotation save.
- Multiple teachers edit same note.

Error states:

- Annotation save failed.
- Unsupported file type.

- Conflict detected.

Empty states:

- No annotation yet: show original note.

Permissions:

- Only teacher/admin can annotate official note.
- Students cannot modify shared note.

Success criteria:

- Important points become clearer.
- Teachers can correct uploaded notes without re-uploading from scratch.

8.8. AI Question Paper Generator

Feature	AI Question Paper Generator
Release phase	MVP strong differentiator

Purpose:

Reduce teacher workload by generating question papers from admin-uploaded syllabus and sample paper patterns.

Detailed flow:

- Teacher opens Generate Question Paper.
- Teacher selects classroom/subject.
- System shows mapped syllabus and available sample papers.
- Teacher selects chapters/topics, question types, marks, duration, difficulty, and blueprint.
- Teacher clicks Generate.
- Backend sends syllabus/sample-paper context to AI workflow.
- Generated paper appears in preview.

User story:

As a teacher, I want AI to generate a question paper from my syllabus and exam pattern so I can save preparation time.

Acceptance criteria:

- Teacher can generate paper only for classrooms with indexed syllabus.
- Teacher can select chapter/topic range.
- Teacher can choose marks, duration, difficulty, and question types.
- Sample papers influence pattern and important question coverage.
- Generated output can be previewed, edited, downloaded, and saved.

UI behavior:

- Step-based wizard: Classroom -> Portion -> Pattern -> Generate -> Preview.
- Clearly show AI sources: syllabus and sample papers mapped.
- Loading state: Generating question paper.
- Show warning: Review before official use.

Backend requirement:

- Question_papers table.
- Syllabus_documents and sample_papers indexed as source documents.
- AI generation service behind Node backend.
- Background job queue for long generation.
- PDF/DOCX export service.
- Usage quota tracking.

Edge cases:

- No syllabus mapped.
- Syllabus indexing pending.
- Teacher selects chapters not found.
- AI output too long/short.
- Repeated important questions.

- Cost/timeout from AI provider.

Error states:

- Generation failed.
- AI service unavailable.
- No source content found.
- Usage limit reached.
- Export failed.

Empty states:

- No syllabus: Ask admin to upload/map syllabus.
- No sample papers: Generate from syllabus only, but show lower-pattern warning.
- No generated papers yet.

Permissions:

- Only teachers assigned to classroom can generate.
- Admin can view/control AI usage.
- Students cannot generate papers in MVP.
- Answer keys visible only to teachers/admin.

Success criteria:

- Teachers can generate usable first draft in minutes.
- Question papers align with syllabus and pattern.
- Teacher workload reduction is clearly measurable.

8.9. Generated paper preview, edit, download, and share

Feature	Generated paper preview, edit, download, and share
Release phase	MVP

Purpose:

Allow teachers to verify and customize AI-generated papers before using them officially.

Detailed flow:

- AI generation completes.
- Teacher sees structured preview.
- Teacher edits title, marks, sections, questions, and instructions.
- Teacher optionally generates answer key.
- Teacher downloads PDF/DOCX or saves to classroom/private library.
- Teacher may share with selected teachers/admin.

User story:

As a teacher, I want to review and edit the generated paper so it is ready for real exam use.

Acceptance criteria:

- Preview is editable before export.
- Teacher can delete/regenerate individual questions or whole sections.
- PDF export preserves formatting.
- Question paper can be saved.
- Answer key handled separately and confidentially.

UI behavior:

- Editor with sections and question cards.
- Actions: Edit, Regenerate, Delete, Move, Download PDF, Save.
- Answer key toggle visible only to teacher/admin.
- Version indicator after edits.

Backend requirement:

- Question_paper_sections/questions tables or JSON structure.
- Version history table.

- Export service.
- File storage for exported PDFs.
- Answer_key confidential field/file.

Edge cases:

- Teacher accidentally closes page.
- AI creates duplicate questions.
- Marks total mismatch.
- Answer key requested after edit.
- Export formatting issue.

Error states:

- Save failed.
- Export failed.
- Regenerate failed.
- Marks validation error.

Empty states:

- No generated papers: Generate your first paper.
- No answer key: Generate answer key CTA.

Permissions:

- Teacher/admin can view answer key.
- Students cannot access draft/generated papers unless teacher publishes student-facing practice paper.
- Admin can audit generated content.

Success criteria:

- Teacher trusts output after editing.
- Exported paper looks professional.
- Confidential content stays protected.

8.10. Teacher-only classroom AI chatbot

Feature	Teacher-only classroom AI chatbot
Release phase	MVP / Phase 1.5

Purpose:

Give each classroom its own AI assistant grounded in the admin-uploaded syllabus and sample papers, initially only for teachers.

Detailed flow:

- Admin uploads/maps syllabus and sample papers.
- AI indexing completes.
- Teacher opens classroom AI Assistant.
- Teacher asks a syllabus or paper-setting question.
- Backend retrieves relevant syllabus/sample-paper chunks.
- AI replies with grounded answer and source references.
- Teacher may use answer to plan lessons/papers.

User story:

As a teacher, I want to chat with an AI assistant trained on my classroom syllabus so I can plan teaching and question papers faster.

Acceptance criteria:

- AI assistant appears only when classroom knowledge base is ready.
- Answers are restricted to mapped syllabus/sample papers unless general-help mode is explicitly enabled.
- Teacher can ask for chapter summary, important topics, question ideas, and teaching plan.
- AI response should cite/mention source document section where possible.
- Chat history is saved per teacher/classroom.

UI behavior:

- Chat screen inside classroom detail.
- Source chips under answers.
- Suggested prompts: Summarize Chapter 3, Generate 5 important questions, Explain topic for class.
- Clear label: Teacher-only AI Assistant.
- Show indexing status if not ready.

Backend requirement:

- RAG pipeline using Flowise/Langflow or custom Node AI service.
- Postgres + pgvector or external vector DB.
- Classroom_knowledge_bases table.
- Chat_sessions and chat_messages tables.
- Source document retrieval with tenant isolation.
- AI guardrails and usage tracking.

Edge cases:

- Syllabus PDF is scanned/poor OCR.
- Teacher asks out-of-syllabus question.
- Sample papers contain old pattern.
- AI hallucination risk.
- Multiple syllabus versions.
- High token/cost usage.

Error states:

- AI assistant not ready.
- Indexing failed.
- No relevant source found.
- AI provider timeout.
- Usage limit reached.
- Unsafe prompt blocked.

Empty states:

- No knowledge base: Ask admin to upload syllabus.
- No chat history: show suggested prompts.
- No answer found: AI should say it cannot find in syllabus.

Permissions:

- Teacher-only in MVP.
- Students cannot access this chatbot.
- Admin can enable/disable and audit usage.
- Strict institution/classroom data isolation required.

Success criteria:

- Teachers get useful syllabus-based help.
- AI answers reduce planning burden.
- No confidential teacher resources leak to students.

8.11. Manage student doubts

Feature	Manage student doubts
Release phase	Phase 2

Purpose:

Help teachers respond to student questions in an organized queue instead of scattered chats.

Detailed flow:

- Teacher opens Doubts tab.
- System lists open doubts by classroom/note/date.
- Teacher opens a doubt thread.
- Teacher replies with text/voice/file.

- Teacher marks as answered/resolved.
- Student receives notification.

User story:

As a teacher, I want all student doubts organized by note and class so I can answer efficiently.

Acceptance criteria:

- Teacher sees pending doubts count.
- Teacher can reply and mark status.
- Doubts link back to note context.
- Teacher can filter by classroom/status.
- Student notified on reply.

UI behavior:

- Inbox-like queue with status badges.
- Thread view with note context at top.
- Quick reply box.
- Mark Resolved button.

Backend requirement:

- Doubts and replies tables.
- Notification event on teacher reply.
- Optional voice/file upload support.
- Status workflow.

Edge cases:

- Many duplicate doubts.
- Student asks unrelated question.
- Teacher assigned to multiple sections.
- Doubt from removed student.

Error states:

- Reply failed.
- Thread load failed.
- Access denied.
- Attachment failed.

Empty states:

- No pending doubts.
- No resolved doubts.

Permissions:

- Teacher sees doubts for assigned classrooms.
- Admin can audit if enabled.
- Students see own/public doubts based on policy.

Success criteria:

- Teacher response time improves.
- Doubt handling becomes contextual and organized.

8.12. Create assignments and quizzes

Feature

Release phase

Create assignments and quizzes

Phase 2

Purpose:

Let teachers create practice tasks and quick checks connected to classroom lessons.

Detailed flow:

- Teacher opens Assignments/Quizzes.

- Teacher chooses assignment or quiz.
- Teacher enters title, instructions, due date, questions/files.
- Teacher selects classroom.
- Teacher publishes.
- Students receive notification.

User story:

As a teacher, I want to assign work from the same platform where students access notes.

Acceptance criteria:

- Teacher can create assignment with due date.
- Teacher can create basic quiz questions.
- Teacher can publish/unpublish.
- Submissions/attempts visible to teacher.
- Students receive notifications.

UI behavior:

- Creation wizard.
- Question editor for MCQ/short answer.
- Due date picker.
- Publish confirmation.

Backend requirement:

- Assignments/quizzes/submissions/attempts tables.
- File upload support.
- Notification scheduling.
- Evaluation logic for objective quizzes.

Edge cases:

- Teacher edits after submissions.
- Late submissions.
- Quiz retake policy.
- Wrong answer key.
- Classroom archived.

Error states:

- Publish failed.
- Invalid due date.
- Missing question options.
- Submission list failed.

Empty states:

- No assignments created.
- No submissions yet.

Permissions:

- Teacher can create for assigned classrooms.
- Students can view/submit only assigned items.
- Admin can monitor.

Success criteria:

- Teachers use Vaasenk beyond notes.
- Student practice and accountability improve.

8.13. Teacher engagement analytics

Feature

Release phase

Teacher engagement analytics

Phase 2

Purpose:

Show teachers whether students are accessing notes, downloading materials, and responding to assignments.

Detailed flow:

- Teacher opens Analytics in classroom.
- System displays note views, downloads, bookmarks, pending students, assignment completion.
- Teacher filters by date/note/student.
- Teacher identifies students who need follow-up.

User story:

As a teacher, I want to know whether students actually viewed the notes so I can support those falling behind.

Acceptance criteria:

- Analytics available per classroom/note.
- Teacher can see aggregate and optionally student-level access based on institution policy.
- Download/view counts are accurate enough for guidance.
- Data export optional.

UI behavior:

- Charts/cards: Views, Downloads, Not Viewed, Doubts, Assignment completion.
- Student list with status.
- Avoid overwhelming charts in MVP.

Backend requirement:

- Events tracking table for view/download/bookmark/submission.
- Aggregated analytics endpoint.
- Privacy settings from admin.
- Background aggregation jobs.

Edge cases:

- Student views offline then syncs later.
- Multiple device views.
- Bot/preload counted as view.
- Privacy policy disables individual tracking.

Error states:

- Analytics load failed.
- Partial data unavailable.
- Export failed.

Empty states:

- No activity yet.
- Analytics disabled by admin.

Permissions:

- Teacher can view assigned classroom analytics.
- Admin controls whether individual student tracking is visible.
- Students do not see class-wide analytics.

Success criteria:

- Teachers identify low-engagement students.
- Analytics helps schools justify subscription value.

8.14. Teacher notifications and reminders

Feature

Release phase

Teacher notifications and reminders

MVP

Purpose:

Keep teachers aware of student joins, doubts, AI job completion, upload success, and pending classroom activity.

Detailed flow:

- Relevant event occurs.
- Teacher receives in-app/push notification.
- Teacher opens notification center.
- Teacher taps notification to open classroom/note/doubt/generated paper.

User story:

As a teacher, I want to be notified when students ask doubts or AI generation completes.

Acceptance criteria:

- Notifications for student joined, new doubt, AI paper ready, upload failed/succeeded, assignment submissions.
- Deep links work.
- Unread count visible.
- Preferences configurable.

UI behavior:

- Bell icon in web/mobile.
- Grouped by classroom.
- Priority labels for doubts/AI completion.
- Read/unread state.

Backend requirement:

- Notifications table.
- Push/email optional integrations.
- Event bus from note upload, joins, AI jobs.
- Device token handling.

Edge cases:

- Many students join at once.
- AI job completes after teacher logs out.
- Notification points to deleted item.
- Teacher disables push.

Error states:

- Notification delivery failed.
- Deep link target unavailable.
- Permission denied for push.

Empty states:

- No notifications: all caught up.

Permissions:

- Teacher receives only assigned classroom events.
- Admin can send school-wide notices.

Success criteria:

- Teachers respond faster.
- AI/job status is not confusing.
- No excessive notification spam.

8.15. Teacher profile and settings

Feature	Teacher profile and settings
Release phase	MVP

Purpose:

Allow teachers to manage account details and teaching preferences.

Detailed flow:

- Teacher opens Profile/Settings.

- Teacher views account, assigned classrooms, preferences.
- Teacher updates allowed fields.
- Teacher changes password/notification settings.
- Teacher logs out.

User story:

As a teacher, I want to manage my profile and notification preferences easily.

Acceptance criteria:

- Profile view works.
- Institution-locked fields are read-only.
- Notification settings save.
- Password change works.
- Logout invalidates session.

UI behavior:

- Profile header with name/role.
- Settings grouped: Account, Notifications, Appearance, Security.
- Clear logout.

Backend requirement:

- Users update endpoint.
- Preferences table/json.
- Auth password change.
- Device token cleanup on logout.

Edge cases:

- Admin controls teacher info.
- Teacher changes phone/email requiring verification.
- Teacher has multiple devices.

Error states:

- Update failed.
- Verification failed.
- Password error.
- Session expired.

Empty states:

- No profile photo: initials avatar.

Permissions:

- Teacher can edit allowed own profile fields.
- Cannot self-upgrade role or access billing.

Success criteria:

- Teacher preferences persist.
- Security/account flows are clear.

8.16. Teacher collaboration and reusable paper library

Feature	Teacher collaboration and reusable paper library
Release phase	Phase 3

Purpose:

Allow teachers in the same institution to reuse approved generated papers, templates, and teaching resources.

Detailed flow:

- Teacher saves paper/resource to private or shared library.
- Another authorized teacher searches shared library.
- Teacher previews item.

- Teacher duplicates/adapts it for own classroom.
- Original remains unchanged.

User story:

As a teacher, I want to reuse good resources from my institution so I do not start from scratch every time.

Acceptance criteria:

- Teacher can mark item private/shared if permitted.
- Shared items are visible only within institution/department.
- Duplicate creates editable copy.
- Answer keys remain protected.

UI behavior:

- Library screen with filters by subject/class/type.
- Share toggle with permission note.
- Duplicate to my classroom action.

Backend requirement:

- Resource_library table.
- Sharing permissions.
- Clone/duplicate service.
- Audit logs.

Edge cases:

- Teacher shares confidential exam by mistake.
- Different syllabus versions.
- Duplicate title conflicts.
- Original deleted after copy.

Error states:

- Share failed.
- Access denied.
- Duplicate failed.
- Resource not found.

Empty states:

- No shared resources yet.
- No search results.

Permissions:

- Admin controls sharing policy.
- Teachers can share only allowed resources.
- Students cannot access teacher library.

Success criteria:

- Institution saves preparation time.
- High-quality resources get reused safely.

9. Admin Side - Detailed Feature Specifications

9.1. Admin authentication and institution dashboard

Feature	Admin authentication and institution dashboard
Release phase	MVP

Purpose:

Give school/college admins a secure control center for users, classrooms, syllabus, billing, AI setup, and analytics.

Detailed flow:

- Admin logs in.
- System validates admin role and institution.
- Admin lands on dashboard.
- Dashboard shows users, classrooms, uploads, AI status, storage, subscription, and alerts.

User story:

As an admin, I want one dashboard to manage the whole institution using Vaasenk.

Acceptance criteria:

- Admin sees admin dashboard only.
- Admin can access institution-level management.
- Subscription and AI status visible.
- Critical alerts visible.
- Unauthorized roles blocked.

UI behavior:

- Web-first dashboard with side navigation.
- Cards for Teachers, Students, Classrooms, Syllabus, AI Jobs, Billing.
- Mobile admin can be limited/responsive but web is primary.

Backend requirement:

- Users role=admin.
- Institution table.
- Admin dashboard aggregation endpoint.
- Role middleware.
- Subscription status service.

Edge cases:

- Multiple admins.
- Admin manages multiple branches.
- Subscription expired.
- Partial AI service outage.

Error states:

- Access denied.
- Dashboard load failed.
- Subscription inactive.
- Session expired.

Empty states:

- New institution: setup checklist.
- No classrooms/users yet.

Permissions:

- Admin can manage institution-level data.
- Superadmin/platform owner permissions separate from school admin.
- Admin cannot access other institutions.

Success criteria:

- Institution setup is clear.

- Admin can understand usage and health at a glance.

9.2. Institution onboarding and profile setup

Feature	Institution onboarding and profile setup
Release phase	MVP

Purpose:

Create the institution workspace with academic metadata, contact details, branding, and subscription mapping.

Detailed flow:

- Platform/admin creates institution or school admin signs up.
- Admin enters institution name, type, board, address, contact, logo.
- Admin selects plan or trial.
- System creates institution workspace.
- Admin sees setup checklist.

User story:

As an admin, I want to set up my school/college workspace so teachers and students use the correct institution environment.

Acceptance criteria:

- Institution can be created with required details.
- Board type can be selected: Samacheer Kalvi, CBSE, Custom, College, Coaching.
- Logo/theme optional.
- Setup checklist guides next steps.
- Institution ID/tenant created.

UI behavior:

- Onboarding wizard with progress steps.
- Fields grouped: Basic, Academic, Contact, Branding, Subscription.
- Save and continue behavior.

Backend requirement:

- Institutions table.
- Institution_settings table/json.
- Subscription mapping.
- Tenant isolation key institution_id across all tables.

Edge cases:

- Institution has multiple boards.
- Multiple branches.
- Wrong board selected.
- Incomplete contact info.
- Trial setup without payment.

Error states:

- Duplicate institution.
- Missing required fields.
- Plan activation failed.
- Logo upload failed.

Empty states:

- First login: setup checklist.
- No branding: use Vaasenk default theme.

Permissions:

- Only admin/superadmin can create/update institution profile.
- Teachers/students read limited institution info only.

Success criteria:

- Admin completes setup without support.
- Tenant isolation starts correctly from day one.

9.3. Subscription and billing management

Feature	Subscription and billing management
Release phase	MVP

Purpose:

Enable recurring monetization at institution level with one payment covering unlimited classrooms under plan limits.

Detailed flow:

- Admin opens Billing.
- Admin views current plan, billing cycle, user/storage/AI limits.
- Admin selects monthly/yearly plan.
- Payment is processed or invoice generated.
- Subscription status updates.
- Admin receives invoice/receipt.

User story:

As an admin, I want one subscription for the whole institution so I do not pay per classroom.

Acceptance criteria:

- Plan details visible.
- Monthly/yearly billing supported.
- Payment status tracked.
- Expired subscription restricts new actions based on policy.
- Invoices accessible.

UI behavior:

- Billing page with plan cards: Starter, Growth, Institution.
- Usage indicators for users/storage/AI quota.
- Renew/upgrade CTA.
- Invoice list.

Backend requirement:

- Subscriptions, invoices, payments tables.
- Payment gateway integration like Razorpay later.
- Webhook handling.
- Plan limits enforcement.
- Grace period logic.

Edge cases:

- Payment succeeds but webhook delayed.
- Failed renewal.
- Plan downgrade with over-limit users.
- Manual offline payment for schools.
- GST/invoice requirements.

Error states:

- Payment failed.
- Invoice generation failed.
- Subscription expired.
- Webhook mismatch.

Empty states:

- No billing history.
- Trial active: show days left.

Permissions:

- Only admin/billing owner can manage billing.
- Teachers/students cannot view billing.
- Platform superadmin can override.

Success criteria:

- Recurring revenue enabled.
- Schools understand pricing.
- Access restriction does not surprise active users.

9.4. Teacher and student user management

Feature	Teacher and student user management
Release phase	MVP

Purpose:

Allow admins to add, invite, edit, deactivate, and manage teachers/students at institution level.

Detailed flow:

- Admin opens Users.
- Admin selects Teachers or Students.
- Admin adds single user or imports bulk CSV.
- System creates accounts/invites.
- Admin assigns roles/classes.
- Users can log in.

User story:

As an admin, I want to manage all teachers and students so access stays organized and secure.

Acceptance criteria:

- Admin can add teacher/student manually.
- Admin can edit/deactivate users.
- Admin can assign classroom/department.
- User list searchable/filterable.
- Bulk import supported in Phase 2 or MVP if needed.

UI behavior:

- User table with filters by role/status/class.
- Add User modal.
- Bulk Import button.
- Status badges: Active, Invited, Inactive.
- Row actions: Edit, Reset Password, Deactivate.

Backend requirement:

- Users table with role/status/institution_id.
- Invitations table.
- Bulk import parser.
- Email/SMS/OTP invite service.
- Password reset flow.

Edge cases:

- Duplicate phone/email.
- Student moves class.
- Teacher leaves institution.
- CSV has invalid rows.
- Parent/guardian data later.

Error states:

- Duplicate user.
- Invalid CSV.

- Invite failed.
- User update failed.
- Cannot deactivate last admin.

Empty states:

- No users: Add first teacher/student.
- No search results.

Permissions:

- Admin manages all institution users.
- Teachers can view students in assigned classrooms only.
- Students cannot view full user directory.

Success criteria:

- User onboarding is smooth.
- Admin can control access without developer help.
- Inactive users cannot access content.

9.5. Bulk import users

Feature	Bulk import users
Release phase	Phase 2, optional MVP for schools

Purpose:

Reduce onboarding effort by importing many teachers/students using CSV/Excel templates.

Detailed flow:

- Admin downloads template.
- Admin fills teacher/student data.
- Admin uploads file.
- System validates rows.
- Admin reviews errors/warnings.
- Admin confirms import.
- System creates accounts/invites and reports results.

User story:

As an admin, I want to upload many users at once so onboarding a school does not take days.

Acceptance criteria:

- Template available.
- Validation preview before import.
- Partial import supported with error report.
- Duplicate handling options.
- Import result downloadable.

UI behavior:

- Import wizard: Upload -> Validate -> Review -> Confirm -> Result.
- Show row-level errors clearly.
- Progress indicator for large imports.

Backend requirement:

- CSV/XLSX parser.
- Validation service.
- Bulk job queue.
- Import_logs table.
- User creation transaction strategy.

Edge cases:

- Large file.
- Wrong columns.

- Duplicate rows.
- Mixed teacher/student file.
- Invalid phone/email.
- Network loss during upload.

Error states:

- File format invalid.
- Too many errors.
- Import job failed.
- Duplicate constraint failure.

Empty states:

- No imports yet: show template download.
- Validation no errors: show ready to import.

Permissions:

- Only admin can import users.
- Imported users limited to admin institution.
- Sensitive data handled securely.

Success criteria:

- Institution onboarding time drops significantly.
- Fewer manual user entry mistakes.

9.6. Academic structure setup

Feature	Academic structure setup
Release phase	MVP

Purpose:

Model school/college structure such as board, class, section, subject, department, academic year, and term.

Detailed flow:

- Admin opens Academic Setup.
- Admin creates academic year.
- Admin adds board/standard/section/subject or department/course/semester.
- Admin maps teachers to subjects/classes.
- System uses structure for classrooms and syllabus mapping.

User story:

As an admin, I want to define our academic structure so classrooms and syllabus mapping are accurate.

Acceptance criteria:

- Admin can create class/section/subject hierarchy.
- Supports school and college models.
- Academic year can be active/archived.
- Teachers can be mapped to subjects/classes.
- Structure reused during classroom creation.

UI behavior:

- Tree or grouped card layout.
- Add Class/Section/Subject modals.
- Academic year selector.
- Mapping table for teachers.

Backend requirement:

- Academic_years, classes, sections, subjects, departments/courses tables.
- Teacher_subject_assignments.
- Institution-specific configuration.

Edge cases:

- School follows CBSE and Samacheer.
- College semester model differs.
- Subject renamed mid-year.
- Teacher teaches same subject multiple sections.

Error states:

- Duplicate subject/class.
- Invalid academic year.
- Cannot delete structure with active classrooms.

Empty states:

- No academic year: Create first academic year.
- No subjects: Add subjects to continue.

Permissions:

- Admin manages structure.
- Teachers may view assigned structure.
- Students only see relevant classroom labels.

Success criteria:

- Classroom creation becomes consistent.
- Syllabus and analytics can map correctly.

9.7. Classroom creation, assignment, and syllabus mapping

Feature	Classroom creation, assignment, and syllabus mapping
Release phase	MVP

Purpose:

Create classroom rooms with assigned teachers/students and mapped syllabus so teachers do not handle syllabus upload repeatedly.

Detailed flow:

- Admin opens Classrooms.
- Admin creates classroom with name, class/section/subject/teacher.
- Admin selects board/syllabus document or uploads new syllabus.
- Admin optionally maps sample papers.
- System creates classroom and join code.
- Teacher sees classroom with AI readiness status.

User story:

As an admin, I want to create classrooms and map the correct syllabus so teachers can start using Vaasenk without setup burden.

Acceptance criteria:

- Admin can create classroom.
- Admin can assign teacher(s).
- Admin can map syllabus repository item.
- Admin can map sample papers.
- Classroom gets unique join code.
- Teacher sees if AI assistant is Ready/Pending/Not configured.

UI behavior:

- Classroom wizard: Details -> Teacher -> Students/Join Code -> Syllabus -> Sample Papers -> Review.
- AI readiness badge.
- Copy join code button.

Backend requirement:

- Classrooms table.

- Classroom_teacher_assignments.
- Classroom_syllabus_mapping.
- Classroom_sample_paper_mapping.
- Join code generation.
- AI indexing trigger.

Edge cases:

- No syllabus available yet.
- Multiple teachers.
- Wrong syllabus mapped.
- Classroom created before students imported.
- Code collision.
- Teacher changes class mid-year.

Error states:

- Classroom creation failed.
- Syllabus mapping failed.
- Teacher assignment failed.
- Code generation failed.

Empty states:

- No classrooms: Create first classroom.
- No syllabus: Upload or use built-in repository later.

Permissions:

- Admin creates/manages classrooms.
- Teachers can create only if institution permits; otherwise admin-controlled.
- Students join only using code or admin assignment.

Success criteria:

- Teachers receive ready-to-use classroom spaces.
- AI features become available without teacher uploading syllabus manually.

9.8. Syllabus repository upload and management

Feature	Syllabus repository upload and management
Release phase	MVP

Purpose:

Centralize syllabus PDFs at admin level for Samacheer Kalvi, CBSE, custom school syllabus, college subjects, and coaching courses.

Detailed flow:

- Admin opens Syllabus Repository.
- Admin uploads syllabus PDF.
- Admin enters board, class, subject, academic year, version, language.
- System validates and stores file.
- AI processing/OCR/chunking starts.
- Admin maps syllabus to classrooms.

User story:

As an admin, I want to upload syllabus once so all relevant classrooms can reuse it.

Acceptance criteria:

- PDF upload supported.
- Metadata required for board/class/subject/year.
- Versioning supported.
- Processing status visible.
- Syllabus can be mapped to multiple classrooms.
- Old versions can be archived.

UI behavior:

- Repository list with filters: Board, Class, Subject, Year, Status.
- Upload modal/wizard.
- Processing badge: Uploaded, Processing, Ready, Failed.
- Preview PDF.

Backend requirement:

- Syllabus_documents table.
- Object storage.
- Document text extraction/OCR.
- Chunking and embedding jobs.
- pgvector/external vector store.
- Version and status fields.

Edge cases:

- Scanned PDF.
- Multiple subjects in one PDF.
- Wrong board/class metadata.
- Syllabus update mid-year.
- Tamil/English mixed text.
- Duplicate syllabus.

Error states:

- Upload failed.
- OCR failed.
- Embedding failed.
- File too large.
- Unsupported/protected PDF.

Empty states:

- No syllabus uploaded: Upload first syllabus.
- No ready syllabus: show processing queue.

Permissions:

- Only admin can upload/manage institution syllabus.
- Teachers can view mapped syllabus and use AI; upload permission optional.
- Students cannot access raw teacher/admin syllabus repository unless published.

Success criteria:

- One syllabus serves many classrooms.
- Teachers avoid repeated uploads.
- AI question generation and chatbot have reliable source material.

9.9. Sample question paper and exam pattern library

Feature	Sample question paper and exam pattern library
Release phase	MVP strong differentiator

Purpose:

Allow admins to upload sample papers so AI generation follows real exam style and important question patterns.

Detailed flow:

- Admin opens Sample Papers.
- Admin uploads paper PDF/DOC/image.
- Admin enters board/class/subject/year/exam type/marks/duration.
- System extracts structure and questions where possible.
- Admin maps sample papers to syllabus/classrooms.
- AI uses them as pattern references during generation.

User story:

As an admin, I want to upload sample question papers so AI does not miss important patterns or frequently asked question styles.

Acceptance criteria:

- Sample paper upload supported.
- Metadata captures exam pattern.
- Paper can be mapped to subjects/classrooms.
- Processing status visible.
- AI generator can use sample papers as reference, not blindly copy.

UI behavior:

- Sample paper library list with metadata.
- Upload and map flow similar to syllabus.
- Preview extracted sections/questions if available.
- Pattern summary card.

Backend requirement:

- Sample_papers table.
- Question_paper_patterns table optional.
- Document extraction/OCR.
- Embedding/indexing.
- Mappings to syllabus/classroom.
- AI prompt/retrieval logic that includes pattern context.

Edge cases:

- Old paper with outdated pattern.
- Poor scan quality.
- Paper contains answer key.
- Multiple exams in one file.
- Copyright/school-confidential papers.

Error states:

- Upload failed.
- Extraction failed.
- Pattern parsing failed.
- Mapping failed.
- AI cannot use sample paper yet.

Empty states:

- No sample papers: AI uses syllabus only.
- No processed papers: show pending status.

Permissions:

- Admin manages sample library.
- Teachers can use mapped sample papers for generation.
- Students cannot view confidential sample papers/answer keys by default.

Success criteria:

- Generated question papers feel closer to real exam standards.
- Teachers trust AI output more.
- Important question coverage improves.

9.10. AI document processing and indexing monitor

Feature	AI document processing and indexing monitor
Release phase	MVP / Phase 1.5

Purpose:

Let admins track whether uploaded syllabi and sample papers are ready for AI chatbot and question generation.

Detailed flow:

- Admin uploads document.
- System starts processing job.
- Admin opens AI Processing Monitor.
- Admin sees status per document/classroom.
- Failed jobs show reason and retry action.
- Ready documents enable AI features.

User story:

As an admin, I want to know whether AI processing is complete so teachers do not face confusing errors.

Acceptance criteria:

- Status visible: Queued, Extracting, Chunking, Embedding, Ready, Failed.
- Retry option for failed jobs.
- Processing logs/errors simplified.
- Classroom AI readiness derived from document status.

UI behavior:

- AI status dashboard with progress badges.
- Retry button.
- Filter by failed/pending/ready.
- Human-readable error reason.

Backend requirement:

- Background queue using BullMQ/Redis.
- Document_processing_jobs table.
- Worker service for OCR/extraction/chunking/embedding.
- Flowise/Langflow API integration or custom pipeline.
- Retry/backoff logic.

Edge cases:

- Worker crash.
- AI provider rate limit.
- PDF text extraction partial.
- Vector store unavailable.
- Multiple versions processing simultaneously.

Error states:

- Processing failed.
- Retry failed.
- Vector DB unavailable.
- OCR service timeout.

Empty states:

- No processing jobs yet.
- All documents ready.

Permissions:

- Admin sees institution-wide processing.
- Teachers see simplified readiness only.
- Students do not see processing internals.

Success criteria:

- Admins can resolve AI setup issues quickly.
- Teachers understand when AI tools are ready.

9.11. Classroom AI bot provisioning and configuration

Feature	Classroom AI bot provisioning and configuration
Release phase	MVP / Phase 1.5

Purpose:

Automatically create or enable teacher-only AI assistants for classrooms with ready syllabus/sample-paper knowledge bases.

Detailed flow:

- Admin maps ready syllabus/sample papers to classroom.
- System creates classroom knowledge base.
- Admin configures AI mode, allowed sources, usage limits, and availability.
- Teacher sees AI Assistant in classroom.
- Admin monitors usage.

User story:

As an admin, I want each classroom to have its own controlled AI assistant so teachers get help from the correct syllabus only.

Acceptance criteria:

- AI assistant can be enabled/disabled per classroom.
- Allowed sources are configurable.
- Teacher-only mode default.
- Usage quota configurable.
- Chat logs/audit policy configurable.
- Status visible.

UI behavior:

- Classroom AI settings panel.
- Toggle: Enable Teacher AI Assistant.
- Source list with selected syllabus/sample papers.
- Usage limit controls.
- Status: Not configured/Pending/Ready.

Backend requirement:

- Classroom_knowledge_bases table.
- AI_bot_settings table.
- Vector namespace per institution/classroom.
- Chat service routes.
- Flowise/Langflow flow IDs if used.
- Usage_metering table.

Edge cases:

- Syllabus changed after bot enabled.
- Multiple syllabi mapped.
- Teacher asks from wrong class.
- Need to re-index after update.
- High usage.

Error states:

- Bot provisioning failed.
- Knowledge base missing.
- AI disabled.
- Usage quota reached.

Empty states:

- No AI configured: map syllabus first.
- Bot ready but no chats yet.

Permissions:

- Admin controls enablement.
- Teachers use bot for assigned classrooms.
- Students blocked in MVP.
- Tenant isolation mandatory.

Success criteria:

- Every eligible classroom has a reliable teacher AI assistant.
- Admin can control cost and access.

9.12. AI usage controls and quota management

Feature

Release phase

AI usage controls and quota management

MVP / Phase 1.5

Purpose:

Protect costs and ensure fair usage for question generation, chatbot, OCR, and embeddings.

Detailed flow:

- Admin opens AI Usage.
- Admin views usage by classroom/teacher/feature.
- Admin sets monthly limits or plan-based defaults.
- System blocks or warns when limits are reached.
- Admin can upgrade plan or reset quota.

User story:

As an admin, I want to control AI usage so costs do not become unpredictable.

Acceptance criteria:

- Usage visible by feature.
- Plan limits enforced.
- Soft and hard limit options.
- Teachers get clear limit messages.
- Admin can export usage report.

UI behavior:

- Usage dashboard with charts/cards.
- Quota settings panel.
- Warning thresholds.
- Upgrade CTA when near limit.

Backend requirement:

- Usage_metering table.
- Token/job count tracking.
- Plan_limits config.
- Middleware before AI calls.
- Alerts when threshold crossed.

Edge cases:

- AI call retries double-counting.
- Multiple providers.
- Manual credits.
- Monthly reset timezone.
- Trial accounts.

Error states:

- Quota reached.
- Usage calculation failed.
- Upgrade/payment failed.

Empty states:

- No usage yet.
- Unlimited plan: show no limit.

Permissions:

- Admin controls quotas.

- Teachers can view own remaining limit if desired.
- Students no AI in MVP.

Success criteria:

- AI costs remain predictable.
- Teachers understand limits before failure.

9.13. Content storage and moderation management

Feature	Content storage and moderation management
Release phase	Phase 2

Purpose:

Help admins monitor storage, remove inappropriate/old content, and manage institution file usage.

Detailed flow:

- Admin opens Storage/Content.
- System lists notes, PDFs, papers, syllabus, sample papers by size/date/classroom.
- Admin filters and reviews.
- Admin archives/deletes content if allowed.
- System updates availability and audit log.

User story:

As an admin, I want to manage uploaded content and storage so the institution stays organized.

Acceptance criteria:

- Storage usage visible.
- Content searchable/filterable.
- Admin can archive/remove content.
- Deletion warnings show affected students/teachers.
- Audit log recorded.

UI behavior:

- Storage usage bar.
- Content table with file type/owner/size/status.
- Archive/Delete actions with confirmation.

Backend requirement:

- File metadata tables.
- Object storage lifecycle rules.
- Soft delete/archive status.
- Audit_logs table.
- Access invalidation for deleted files.

Edge cases:

- Teacher deletes by mistake.
- Student has offline copy.
- Content linked to generated paper.
- Legal retention requirement.

Error states:

- Delete failed.
- Storage API failed.
- Content still referenced.
- Permission denied.

Empty states:

- No uploaded content yet.
- No results for filters.

Permissions:

- Admin can moderate institution content.
- Teachers can manage own uploads depending policy.
- Students cannot delete official materials.

Success criteria:

- Storage remains under control.
- Admins can remove incorrect material safely.

9.14. School-wide announcements

Feature	School-wide announcements
Release phase	Phase 2

Purpose:

Allow admins to send official institution-level announcements to teachers, students, or selected classes.

Detailed flow:

- Admin opens Announcements.
- Admin creates message.
- Admin chooses audience: all, teachers, students, class/section.
- Admin schedules or sends now.
- Recipients receive notification and see announcement.

User story:

As an admin, I want to send official notices without using external WhatsApp groups.

Acceptance criteria:

- Admin can target audience.
- Announcements have title/body/optional attachment.
- Scheduled send optional.
- Read status optional.
- Recipients see announcements in app.

UI behavior:

- Announcement composer.
- Audience selector.
- Preview before send.
- Sent announcements list.

Backend requirement:

- Announcements table.
- Announcement_recipients table.
- Notification dispatch.
- Scheduler for timed sends.

Edge cases:

- Wrong audience selected.
- Urgent notice needs immediate delivery.
- Attachment too large.
- User disabled push.

Error states:

- Send failed.
- Schedule failed.
- Audience empty.
- Attachment upload failed.

Empty states:

- No announcements yet.

- Recipient: no announcements.

Permissions:

- Admin can send institution-wide.
- Teachers may send classroom announcements if allowed.
- Students read only.

Success criteria:

- Official communication becomes centralized.
- Important updates are not lost in chat apps.

9.15. Institution analytics and reports

Feature	Institution analytics and reports
Release phase	Phase 2

Purpose:

Give admins measurable proof of Vaasenk usage, adoption, and learning workflow impact.

Detailed flow:

- Admin opens Analytics.
- System shows usage: active users, notes uploaded, AI generations, downloads, classroom activity, storage, unanswered doubts.
- Admin filters by date/class/teacher/subject.
- Admin exports report.

User story:

As an admin, I want analytics so I can see whether teachers and students are actually using Vaasenk.

Acceptance criteria:

- Active users and feature usage visible.
- Teacher/classroom-level reports available.
- AI usage shown separately.
- Export CSV/PDF optional.
- Data respects privacy settings.

UI behavior:

- Dashboard cards and simple charts.
- Filters at top.
- Export button.
- Alert cards for low activity.

Backend requirement:

- Analytics event tracking.
- Aggregation jobs.
- Reports endpoint.
- Export service.
- Privacy configuration.

Edge cases:

- Offline usage sync late.
- Large institution data volume.
- Teacher privacy concerns.
- Deleted users historical data.

Error states:

- Analytics unavailable.
- Export failed.
- Partial data warning.

Empty states:

- No data yet: Usage appears after activity.
- No results for filters.

Permissions:

- Admin sees institution analytics.
- Teachers see assigned classroom analytics.
- Students do not see admin reports.

Success criteria:

- Schools can justify subscription.
- Admins can identify low adoption and train teachers.

9.16. Roles and permissions management

Feature	Roles and permissions management
Release phase	MVP

Purpose:

Control what admins, teachers, students, and future roles can see and do.

Detailed flow:

- Admin opens Roles/Permissions.
- Admin views default roles.
- Admin configures optional permissions like teacher classroom creation or student uploads if supported.
- System applies permissions across frontend and backend.
- Changes are audited.

User story:

As an admin, I want permissions to be clear so students do not access teacher-only tools.

Acceptance criteria:

- Default roles exist: Admin, Teacher, Student.
- Permission checks happen backend-side.
- UI hides unavailable actions.
- Optional toggles are institution-scoped.
- Permission changes are logged.

UI behavior:

- Permissions table with simple toggles.
- Warning text for risky permissions.
- Read-only defaults where not configurable.

Backend requirement:

- RBAC middleware.
- Permissions table or role_permissions JSON.
- Frontend route guards.
- Audit logs for changes.

Edge cases:

- Teacher promoted to admin.
- Multiple admins with different rights.
- Cached permissions after change.
- Student tries direct API call.

Error states:

- Permission save failed.
- Access denied.
- Cannot remove own critical admin permission.

Empty states:

- No custom permissions: use defaults.

Permissions:

- Admin controls institution permissions.
- Backend always enforces permissions, not only UI.

Success criteria:

- No unauthorized feature access.
- Clear separation between student/teacher/admin side.

9.17. Audit logs

Feature	Audit logs
Release phase	Phase 2

Purpose:

Maintain accountability for sensitive actions such as uploads, deletes, AI generation, permission changes, and billing updates.

Detailed flow:

- A sensitive action occurs.
- Backend records actor, action, target, timestamp, IP/device metadata where appropriate.
- Admin opens Audit Logs.
- Admin filters and reviews records.
- Admin exports if needed.

User story:

As an admin, I want audit logs so the institution can track important actions and resolve disputes.

Acceptance criteria:

- Sensitive actions logged.
- Logs searchable/filterable.
- Logs are tamper-resistant at application level.
- Export optional.
- Retention policy configurable.

UI behavior:

- Audit log table with filters: actor, action, module, date.
- Readable action names.
- Details drawer.

Backend requirement:

- Audit_logs table.
- Middleware/event logger.
- Retention policy.
- Indexing for search.
- IP/user agent capture if allowed.

Edge cases:

- High log volume.
- Privacy concerns.
- User deleted but logs retained.
- Failed actions also logged.

Error states:

- Log query failed.
- Export failed.

Empty states:

- No logs yet: Actions will appear here.

Permissions:

- Admins/superadmins only.
- Teachers/students cannot view institution audit logs.

Success criteria:

- Accountability improves.
- Admin can diagnose mistaken deletes or permission changes.

9.18. Branding and institution settings

Feature	Branding and institution settings
Release phase	Phase 2

Purpose:

Let institutions configure basic branding, academic preferences, notification rules, and app policies.

Detailed flow:

- Admin opens Settings.
- Admin uploads logo and selects theme if allowed.
- Admin configures academic year, notification defaults, download permissions, AI settings.
- Settings save and apply across users.

User story:

As an admin, I want Vaasenk to match our institution policies and basic branding.

Acceptance criteria:

- Logo/theme optional.
- Download policy configurable.
- Notification defaults configurable.
- AI feature toggles configurable.
- Settings apply institution-wide.

UI behavior:

- Settings grouped: Branding, Academic, Notifications, Downloads, AI, Security.
- Preview branding where possible.

Backend requirement:

- Institution_settings table/json.
- Feature flags.
- Policy enforcement in services.
- Logo storage.

Edge cases:

- Invalid logo size.
- Conflicting settings.
- Policy change affects existing downloads.
- Admin disables AI while teacher is using it.

Error states:

- Settings save failed.
- Logo upload failed.
- Invalid configuration.

Empty states:

- Default Vaasenk branding/settings.

Permissions:

- Admin manages institution settings.
- Teachers/students see applied settings but cannot modify global policy.

Success criteria:

- Schools can adapt Vaasenk without custom development.
- Policies are consistent across web/mobile.

9.19. Data export and backup

Feature	Data export and backup
Release phase	Phase 3

Purpose:

Allow institutions to export important data for compliance, migration, backup, or academic records.

Detailed flow:

- Admin opens Data Export.
- Admin chooses data type: users, classrooms, notes metadata, analytics, generated papers.
- Admin selects date range.
- System prepares export job.
- Admin downloads file when ready.

User story:

As an admin, I want to export institution data so we are not locked in and can keep records.

Acceptance criteria:

- Export request can be created.
- Large exports run in background.
- Download link expires.
- Export action audited.
- Sensitive data export permissions controlled.

UI behavior:

- Export wizard.
- Job status list.
- Download button when ready.
- Warning about sensitive data.

Backend requirement:

- Export_jobs table.
- Background job worker.
- CSV/ZIP/PDF packaging.
- Secure signed download URL.
- Audit log.

Edge cases:

- Very large data.
- Export includes confidential answer keys.
- Admin leaves before job finishes.
- Expired download link.

Error states:

- Export failed.
- No data in range.
- Download expired.
- Permission denied.

Empty states:

- No exports yet.

- No data for selected filters.

Permissions:

- Only authorized admins.
- Sensitive exports require high-level admin/superadmin if needed.

Success criteria:

- Institution trust increases.
- Data portability concerns reduced.

9.20. Support and onboarding checklist

Feature	Support and onboarding checklist
Release phase	MVP

Purpose:

Guide admins through setup and reduce confusion during first institution rollout.

Detailed flow:

- Admin first logs in.
- System shows checklist: institution profile, academic structure, users, classrooms, syllabus, sample papers, subscription.
- Admin completes tasks.
- Checklist updates automatically.
- Admin can access help docs/contact support.

User story:

As an admin, I want a setup checklist so I do not miss important steps before teachers start using Vaasenk.

Acceptance criteria:

- Checklist visible for new institutions.
- Each task links to correct screen.
- Completion updates automatically.
- Dismiss/return allowed.
- Help/support link visible.

UI behavior:

- Setup checklist card on dashboard.
- Progress bar.
- Task links with status icons.
- Help center/contact support CTA.

Backend requirement:

- Setup_state derived from existing data.
- Optional checklist_status table.
- Support/contact form endpoint.
- Help documentation links.

Edge cases:

- Admin completes steps out of order.
- Skipped syllabus but wants notes only.
- Support request after failed setup.
- Multiple admins.

Error states:

- Checklist load failed.
- Support request failed.

Empty states:

- All setup complete: show success and hide/minimize checklist.

Permissions:

- Admins only.
- Teachers may see limited getting-started guide for assigned classrooms.

Success criteria:

- Faster onboarding.
- Lower support burden.
- Institutions reach first successful classroom quickly.

10. Simplified Permissions Matrix

Capability	Student	Teacher	Admin	Notes
Login	Yes	Yes	Yes	All roles use role-based routing.
Join classroom	Yes	No	Can assign users	Students join by code or assignment.
Create classroom	No	Optional by policy	Yes	Recommended admin-owned for MVP.
Upload official notes	No	Yes	Yes	Students can only view unless contributor mode is added.
View notes	Own classrooms	Assigned classrooms	Institution-wide	Access controlled by membership and role.
Upload syllabus	No	Optional future	Yes	Admin-owned to reduce teacher burden.
Upload sample papers	No	Optional future	Yes	Admin-owned for pattern control.
Generate question paper	No	Yes	Yes/monitor	Teacher-only generation in MVP.
View answer key	No	Yes	Yes	Confidential teacher/admin content.
Use AI classroom chatbot	No in MVP	Yes	Configure/monitor	Student AI later after safety controls.
Ask doubt	Yes	Reply/manage	Audit optional	Phase 2.
Manage billing	No	No	Yes	Institution-level subscription.
View analytics	Own activity optional	Assigned classes	Institution-wide	Privacy policy controls detail level.
Manage users	No	Limited view	Yes	Teachers see students in assigned classrooms only.
Audit logs	No	No	Yes	Sensitive admin-only capability.

11. Backend Planning Notes

11.1 Core database entities

- institutions, institution_settings, subscriptions, invoices, payments
- users, user_preferences, invitations, roles, permissions
- academic_years, classes, sections, subjects, departments, teacher_subject_assignments
- classrooms, classroom_members, classroom_teacher_assignments, classroom_syllabus_mapping, classroom_sample_paper_mapping
- notes, note_attachments, note_tags, note_versions, bookmarks, downloads
- syllabus_documents, sample_papers, document_processing_jobs, classroom_knowledge_bases
- question_papers, question_paper_sections, question_paper_questions, generated_paper_exports

- ai_bot_settings, chat_sessions, chat_messages, usage_metering
- doubts, doubt_replies, assignments, submissions, quizzes, quiz_attempts
- notifications, announcements, analytics_events, audit_logs, export_jobs

11.2 Recommended API groups

- /auth - login, refresh, logout, forgot password, verify OTP/email
- /admin/institution - profile, settings, billing, academic setup
- /admin/users - user CRUD, imports, invites, deactivation
- /admin/classrooms - classroom creation, assignment, syllabus/sample mapping
- /admin/syllabus - upload, list, process, map syllabus
- /admin/sample-papers - upload, list, process, map sample papers
- /teacher/classrooms - teacher classroom list/details
- /teacher/notes - upload, edit, publish, archive notes
- /teacher/question-papers - generate, preview, edit, export, save papers
- /teacher/ai-chat - teacher-only classroom AI chatbot
- /student/classrooms - join classroom, list classrooms, classroom feed
- /student/notes - list, view, bookmark, download, search notes
- /notifications - list, mark read, preferences
- /analytics - teacher/admin reporting with permission checks

12. QA and Test Case Guidance

- For every feature, create tests for success, permission denial, validation errors, empty state, and network failure.
- Test direct API access attempts by wrong roles, not only hidden UI buttons.
- Test tenant isolation: user from Institution A must never access Institution B data.
- Test AI features when syllabus is missing, indexing is pending, indexing failed, and usage limit is reached.
- Test file upload with large files, invalid types, protected PDFs, failed storage, and slow network.
- Test mobile offline behavior for cached notes, expired sessions, and deleted/updated files.

13. Suggested Build Order

Phase	Build items	Reason
Phase 1 - Core MVP	Auth, roles, admin institution setup, classroom creation, teacher note upload, student note feed, bookmarks, notifications.	Proves core promise: board to phone, less copying.
Phase 1.5 - AI Differentiator	Admin syllabus/sample paper upload, AI processing monitor, teacher question paper generator, teacher-only classroom chatbot.	Makes Vaasenk hard to compare with WhatsApp/Discord.
Phase 2 - Engagement	Doubts, assignments, quizzes, OCR search, offline downloads, analytics.	Increases daily usage and retention.
Phase 3 - Scale	Parent access, student AI assistant, teacher collaboration library, data export, advanced reports.	Enterprise value and expansion.